LangChain

Chains Handbook



1. chains overview



\

Contenido

0. Chains Overview — LCEL, the | Operator, and Runnables

1. Simple Definition
2. Why Do Chains Exist?

Before chains (manual wiring)

...now repeat for the next step, passing text along by hand 🤩

After chains (LCEL)

3. Real-Life Analogy
4. The Foundation: the Runnable Interface
5. Internal Working — how | builds a chain
6. The building blocks you'll compose

The | (pipe) operator

RunnableSequence (the "sequential" engine)

RunnableParallel (the "parallel" engine)

RunnableBranch (the "conditional" engine)

RunnableLambda & RunnablePassthrough (the glue)

7. The four shapes at a glance

0. Chains Overview — LCEL, the `|` Operator, and Runnables

Before the four chain shapes, let's understand **what a chain actually is**, the **Runnable interface** that makes chains possible, and the `|` **(pipe) operator** that builds them.

---

1. Simple Definition

> ***Kid version:*** *Think of a* ***toy marble run****. You drop a marble in the top, it rolls through one piece, drops into the next piece, then the next, and pops out the bottom. A* ***chain*** *is a marble run for data: your input rolls through step after step and comes out transformed.*

Professional definition: A *chain* is a composition of components (prompts, models, parsers, functions) where each component's output is passed as the next component's input. In modern LangChain, chains are built with **LCEL** by connecting **Runnables** using the `|` operator.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("Tell me a joke about {topic}")
chain = prompt | model | StrOutputParser()      # a 3-step chain
chain.invoke({"topic": "cats"})                  # "Why did the cat..."
```

---

2. Why Do Chains Exist?

The problem: Real tasks need **more than one step**. "Summarize this document, then translate the summary, then extract keywords" is three model calls plus glue. Doing that by hand means manually passing outputs around, handling batching, streaming, and async for each piece.

Before chains (manual wiring)

```
prompt_value = prompt.format_messages(topic="cats")   # 1) format
ai_message = model.invoke(prompt_value)                # 2) call model
text = ai_message.content                              # 3) unwrap
# ...now repeat for the next step, passing `text` along by hand 🤩
```

After chains (LCEL)

```
chain = prompt | model | StrOutputParser()
chain.invoke({"topic": "cats"})     # all three steps, one call
```

Chains give you **composition** (snap steps together), plus **streaming, batching, async, and retries** for free on the whole pipeline — because every piece shares one interface.

Where you'll use it: literally everywhere in LangChain — RAG, agents, extraction, summarization, routing. Chains are the backbone.

---

3. Real-Life Analogy

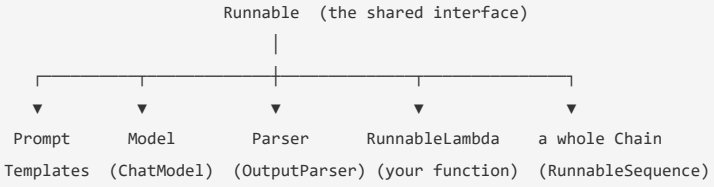
A factory assembly line 🏭. Raw materials enter at one end. Station 1 shapes the part, passes it to Station 2 which paints it, then Station 3 packages it. Each station does one job and hands its result to the next. A chain is that line; each Runnable is a station.

Other analogies: a **relay race** (baton = data passed runner to runner), **Unix pipes** (`cat file | grep x | sort`), a **kitchen line** (prep → cook → plate).

---

4. The Foundation: the Runnable Interface

Every LangChain building block — prompts, models, parsers, even your own functions and whole chains — is a **Runnable**. They all share the same methods:



The core methods every Runnable has:

Method	What it does
`.invoke(input)`	Run on **one** input, return the output.
`.batch([...])`	Run on **many** inputs (parallelized).
`.stream(input)`	Run and **yield** output chunks as they're produced.
`.ainvoke` / `.abatch` / `.astream`	Async versions of the above.

This is the key idea: because *everything* speaks this same interface, you can plug any Runnable into any other. A chain is just a Runnable built from Runnables — so chains nest inside chains.

---

5. Internal Working — how `|` builds a chain

```
prompt | model | parser
      |
      ▼
The `|` operator calls prompt.__or__(model) → RunnableSequence(prompt, model)
      |
      ▼
... | parser → RunnableSequence(prompt, model, parser)
      |
      ▼
A single RunnableSequence Runnable with 3 steps
      |
      ▼
.invoke({"topic": "cats"}):
   step output → next step input, in order:
      {"topic":"cats"} → prompt → PromptValue → model → AIMessage → parser → "..."
```

So `|` doesn't run anything immediately — it **composes** a new Runnable. Execution happens when you call `.invoke()` / `.stream()` / `.batch()`. The output type of each step must match (roughly) the input type the next step expects — that's the one rule to keep in mind.

```
type(prompt | model | parser)   # <class 'langchain_core.runnables.base.RunnableSequence'>
```

---

6. The building blocks you'll compose

The `|` (pipe) operator

Definition: Connects two Runnables so the left's output feeds the right's input.

Why it exists: Readable, declarative composition (reads left-to-right like a pipeline).

When developers use it: To build essentially every chain.

```
chain = prompt | model | StrOutputParser()
```

RunnableSequence (the `"sequential"` engine)

Definition: The Runnable that runs steps **in order**, piping each output to the next. `|` creates it for you.

Why it exists: It's the backbone of simple and sequential chains.

```
from langchain_core.runnables import RunnableSequence
chain = RunnableSequence(prompt, model, parser)   # same as prompt | model | parser
```

RunnableParallel (the `"parallel"` engine)

Definition: Runs multiple Runnables on the **same input at once**, returning a dict of results.

Why it exists: Fan-out work that doesn't depend on each other.

```
from langchain_core.runnables import RunnableParallel
RunnableParallel(joke=joke_chain, poem=poem_chain)
```

RunnableBranch (the `"conditional"` engine)

Definition: Picks **one** of several branches based on a condition (like if/elif/else).

Why it exists: Route different inputs to different sub-chains.

```
from langchain_core.runnables import RunnableBranch
RunnableBranch((is_math, math_chain), (is_code, code_chain), default_chain)
```

`RunnableLambda & RunnablePassthrough` (the glue)

Definition: `RunnableLambda` wraps any Python function as a Runnable; `RunnablePassthrough` passes input through unchanged (often to keep original data alongside new results).

Why they exist: Custom logic and data-shaping between steps.

```
from langchain_core.runnables import RunnableLambda, RunnablePassthrough
```

---

7. The four shapes at a glance

```
SIMPLE          prompt ─► model ─► parser
                        one straight pipeline

SEQUENTIAL      chainA ─► chainB ─► chainC
                        output of each stage feeds the next (multi-stage)

PARALLEL               ┌─► summary ─┐
                input ─┤            ├─► {summary, keywords, sentiment}
                       ├─► keywords ┤
                       └─► sentiment┘   branches run together, merged into a dict
```

```
CONDITIONAL     input ─► router ─► if math   → math_chain
                                   elif code → code_chain
                                   else      → general_chain
```

All four are **just Runnables**, so you can nest them: a parallel step inside a sequential chain inside a conditional branch, etc.

2. simple chain



21 de septiembre de 2026

Contenido

1. The Simple Chain (prompt | model | parser)

1. Simple Definition
2. Why Does It Exist?
   - Before (manual, three separate calls)
   - After (one chain)
3. Real-Life Analogy
4. Where It Fits in LangChain Architecture
5. Internal Working — data flowing through
6. The three parts
   - PromptTemplate (step 1)
   - ChatModel (step 2)
   - OutputParser (step 3)
7. It's a Runnable — so you get everything for free
8. Can a chain be even simpler?

prompt | model → returns the raw AIMessage (no parser)

model | parser → skip templating, feed messages directly

A quick note on the legacy LLMChain

Legacy (deprecated) — shown only so you recognize it:

from langchain.chains import LLMChain

chain = LLMChain(llm=model, prompt=prompt)

Modern equivalent:

1. The Simple Chain (prompt | model | parser)

The most fundamental chain — and the one you'll write most often. Three steps: build a prompt, call the model, parse the output. Master this and every bigger chain is just more of the same.

1. Simple Definition

> ***Kid version:*** *It's a three-step sandwich machine. You put in a filling (your topic), the machine* ***writes the order*** *(prompt),* ***makes the sandwich*** *(model), and* ***wraps it neatly*** *(parser). Out comes a ready sandwich (clean text).*

Professional definition: A simple chain is a `RunnableSequence` of three stages — a **prompt template** (formats input into a prompt), a **chat model** (generates a response), and an **output parser** (converts the model's message into a usable Python value) — connected with `|`.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("Write a one-line tip about {topic}.")
chain = prompt | model | StrOutputParser()

chain.invoke({"topic": "sleep"})     # "Keep a consistent bedtime to improve sleep quality."
```

2. Why Does It Exist?

The problem: Even the simplest LLM call has three chores: turn your variables into a prompt, call the model, and pull clean data out of the model's message object. Doing them separately is verbose and repetitive.

Before (manual, three separate calls)

```
messages = prompt.format_messages(topic="sleep")   # 1) format
ai_msg = model.invoke(messages)                     # 2) call model
text = ai_msg.content                               # 3) unwrap .content
```

After (one chain)

```
chain = prompt | model | StrOutputParser()
text = chain.invoke({"topic": "sleep"})             # all three, one call
```

The chain packages the three chores into one reusable object that also supports **streaming, batching, and async** automatically.

---

3. Real-Life Analogy

A coffee shop order . You say a flavor (input). The barista **writes the order on a cup** (prompt), **makes the drink** (model), and **puts a lid on it** (parser) so you can actually carry it. Each role is distinct, and together they turn "vanilla" into a drinkable coffee.

---

4. Where It Fits in LangChain Architecture

```
{"topic": "sleep"}            ← dict input
     │
     ▼
PromptTemplate   ──→ PromptValue / messages
     │
     ▼
ChatModel        ──→ AIMessage (has .content, maybe .tool_calls)
     │
     ▼
OutputParser     ──→ clean Python value (str, list, dict, object)
     │
     ▼
final output
```

This prompt → model → parser trio is the atom of LangChain. Sequential, parallel, and conditional chains are built by combining these atoms.

---

5. Internal Working — data flowing through

```
chain = prompt | model | parser
chain.invoke({"topic": "sleep"})

① {"topic": "sleep"}
      │  prompt fills the template
      ▼
② ChatPromptValue → [HumanMessage("Write a one-line tip about sleep.")]
      │  model generates
      ▼
③ AIMessage(content="Keep a consistent bedtime...")
      │  parser extracts
      ▼
④ "Keep a consistent bedtime..."   ← a plain str, ready to use
```

Notice how the **type changes** at each step: dict → messages → AIMessage → str. Each stage's output type must fit the next stage's expected input — the trio above is the canonical, always-valid flow.

---

6. The three parts

`PromptTemplate` (step 1)

Definition: Turns input variables into a formatted prompt/messages.

Why it exists: Separates the reusable prompt *shape* from the *values* you fill in.

When developers use it: The entrance of almost every chain.

Real-life use case: Writing the order on the cup.

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a concise assistant."),
    ("human", "Write a one-line tip about {topic}."),
])
```

`ChatModel` (step 2)

Definition: The LLM that generates a response message.

Why it exists: It's the actual intelligence producing the answer.

When developers use it: The middle of every chain.

Real-life use case: The barista making the drink.

```
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
```

`OutputParser` (step 3)

Definition: Converts the model's `AIMessage` into a usable value (string, list, dict, Pydantic object).

Why it exists: Your program wants clean data, not a message object. (See your Output Parser notes.)

When developers use it: Whenever you need the result in a specific shape; `StrOutputParser` is the most common (just the text).

Real-life use case: Putting a lid on so you can carry the coffee.

```
from langchain_core.output_parsers import StrOutputParser
StrOutputParser()          # AIMessage → its .content string
```

---

7. It's a Runnable — so you get everything for free

Because the chain is itself a Runnable, all these work out of the box:

```
chain.invoke({"topic": "sleep"})                # one input

chain.batch([{"topic": "sleep"}, {"topic": "water"}])   # many inputs, parallelized

for chunk in chain.stream({"topic": "sleep"}):   # stream tokens as they arrive
    print(chunk, end="", flush=True)

await chain.ainvoke({"topic": "sleep"})          # async
```

You wrote three components with `|`, and got batching, streaming, and async on the whole pipeline.

---

8. Can a chain be even simpler?

Yes — a chain doesn't *require* all three parts. Two-step chains are common:

```
# prompt | model  → returns the raw AIMessage (no parser)
(prompt | model).invoke({"topic": "sleep"}).content

# model | parser  → skip templating, feed messages directly
```

But `prompt | model | parser` is the standard, complete simple chain — start here.

A quick note on the legacy `LLMChain`

Old tutorials use `LLMChain(llm=model, prompt=prompt)`. It did roughly what `prompt | model` does. It **still works but is deprecated** — prefer LCEL (`|`) for new code. Recognize it in old code; don't write it.

```
# Legacy (deprecated) — shown only so you recognize it:
# from langchain.chains import LLMChain
# chain = LLMChain(llm=model, prompt=prompt)

# Modern equivalent:
chain = prompt | model | StrOutputParser()
```

In [2]:

```
from langchain_ollama import ChatOllama
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

In [3]:

```
# step1: create the prompt template
prompt = PromptTemplate(template='Generate 5 interesting facts about {topic}',
                        input_variables=['topic'])
```

In [4]:

```
# step2: Initialize the language model
llm = ChatOllama(model="qwen3:8b")
```

In [5]:

```
# step3: Initialize output parser
output_parser = StrOutputParser()
```

In [6]:

```
# step4: Create the chain
chain = prompt | llm| output_parser
```

In [7]:

```
# invoke the chain with a topic
result = chain.invoke({'topic':'cricket'})
```

In [8]:

```
print(result)
```

Out [8]:

```
1. **The First Cricket Club**: The Hambledon Cricket Club, founded in 1760 in England, is widely regarded as the first organized cricket club. It played a pivotal role in formalizing rules and promoting the sport, laying the groundwork for modern cricket.

2. **Test Match Origins**: The term "Test Match" comes from the 1877 Ashes series between England and Australia. It was introduced to describe matches that tested players' skills over long periods, eventually becoming the longest format of the game (five days).

3. **The Cricket Ball's Unique Design**: A standard cricket ball is made of leather with a cork core and is stitched with 86 pieces of leather. The seams are critical for seam bowlers, who exploit them to create movement and variation in the ball's flight.

4. **The Duckworth-Lewis Method**: This mathematical formula, developed in the 1990s, is used to calculate fair targets in rain-affected limited-overs matches. It ensures matches remain competitive even when interrupted, balancing the number of overs bowled by each team.

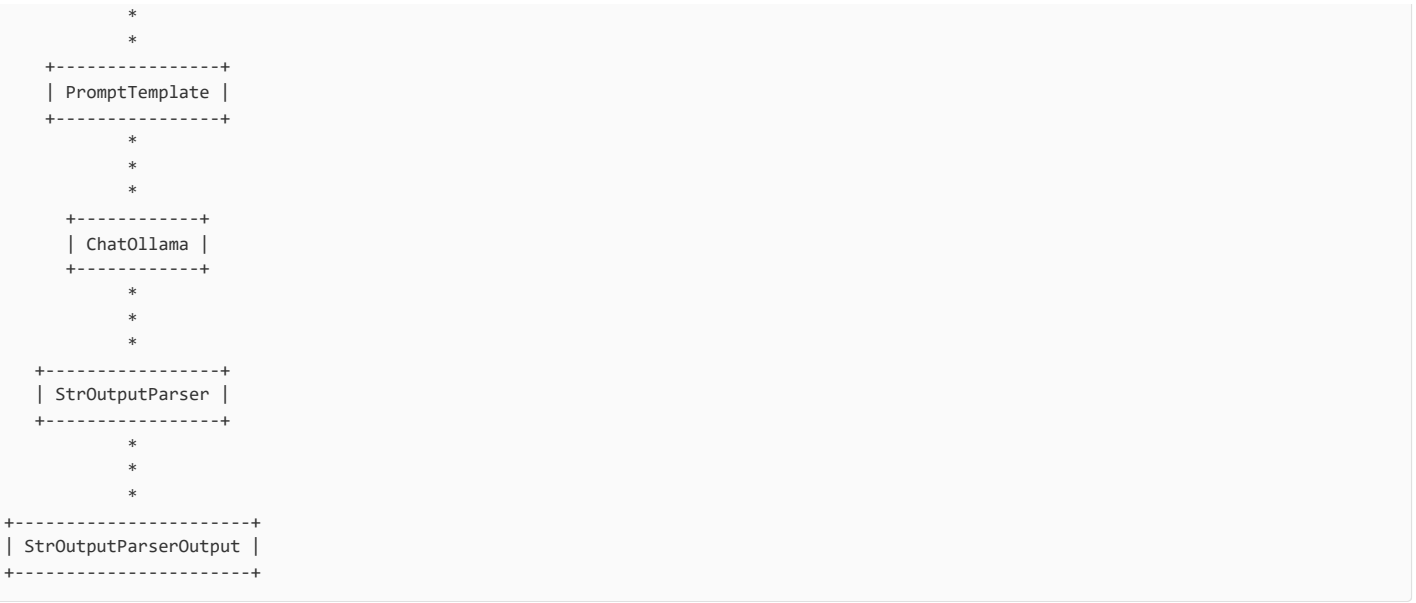
5. **The Pitch's Precision**: A cricket pitch is 22 yards (20.12 meters) long and 10 feet (3.05 meters) wide. The bowling crease is marked at the ends, and the stumps are placed exactly 11 yards apart, creating a precise and strategic playing area.
```

In [9]:

```
chain.get_graph().print_ascii()
```

Out [9]:

```
     +-------------+
     | PromptInput |
     +-------------+
            *
```


*
*
+----------------+
| PromptTemplate |
+----------------+
*
*
*
+------------+
| ChatOllama |
+------------+
*
*
*
+-----------------+
| StrOutputParser |
+-----------------+
*
*
*
+----------------------+
| StrOutputParserOutput |
+----------------------+

3. sequential chain



21 de septiembre de 2026

Contenido

2. The Sequential Chain

1. Simple Definition

Stage 1: summarize → Stage 2: translate the summary to French

2. Why Does It Exist?

Before (manual passing between stages)

You're manually shuttling outputs into the next call. Error-prone, verbose.

After (one sequential chain)

3. Real-Life Analogy

4. Where It Fits in LangChain Architecture

5. Internal Working — the baton passing

Reshape a plain string into the dict the next prompt expects:

6. Building a sequential chain — worked example

Stage 1: make a summary from {text}

Stage 2: translate whatever comes in ({summary}) to French

Glue: stage 1 outputs a str; stage 2's prompt wants {"summary": ...}

→ French translation of the 2-sentence summary

7. Key parts / patterns

Ordered steps (the sequence)

Glue between stages (shape-matching)

Keeping earlier values with RunnablePassthrough.assign (preview)

8. The legacy classes (recognize, don't write)

Legacy (deprecated), shown only for recognition:

from langchain.chains import SimpleSequentialChain

overall = SimpleSequentialChain(chains=[summarize, translate])

Modern:

2. The Sequential Chain

A **sequential chain** is a simple chain grown up: **multiple stages in a row**, where each stage's output becomes the next stage's input. Summarize → translate → extract keywords. It's the same `|` idea, just longer.

1. Simple Definition

> ***Kid version:*** *It's a relay race 🏃‍♂️➡️🏃‍♂️. Runner 1 finishes and hands the baton to Runner 2, who runs and hands it to Runner 3. Each runner starts where the last one left off. In a sequential chain, the "baton" is the data.*

Professional definition: A sequential chain is a `RunnableSequence` of two or more sub-chains/steps executed **in order**, where the output of step *N* is passed as the input to step *N+1*. It models multi-stage workflows that must happen one after another because each stage **depends on** the previous one's result.

```
# Stage 1: summarize → Stage 2: translate the summary to French
chain = summarize_chain | translate_chain
chain.invoke({"text": long_article})     # → French summary
```

2. Why Does It Exist?

The problem: Many tasks are **multi-step and dependent**: you can't translate a summary before you *have* the summary. You need stage 2 to receive stage 1's output.

Before (manual passing between stages)

```
summary = summarize_chain.invoke({"text": article})      # step 1
translated = translate_chain.invoke({"summary": summary}) # step 2 (feed step 1's output by hand)
keywords = keyword_chain.invoke({"text": translated})     # step 3
# You're manually shuttling outputs into the next call. Error-prone, verbose.
```

After (one sequential chain)

```
chain = summarize_chain | translate_chain | keyword_chain
chain.invoke({"text": article})     # all three stages, output auto-piped
```

Sequential chaining automates the hand-off and gives the whole multi-stage pipeline streaming/batching/ async.

Where you'll use it: summarize-then-analyze, generate-then-critique-then-refine, extract-then- validate, translate pipelines, any multi-step reasoning workflow.

---

3. Real-Life Analogy

An assembly line with dependent stations 🏭. The painter can't paint a part until the molder has shaped it; the packer can't pack it until it's painted. Each station **must** wait for the previous one — that ordered dependency is exactly a sequential chain.

Other analogies: a **recipe** (chop → cook → season → plate, in order), an **essay process** (outline → draft → edit).

---

4. Where It Fits in LangChain Architecture

```
RunnableSequence  (the sequential engine — same one `|` builds)
     │
     ▼
step1 ─► step2 ─► step3 ─► ...      each step is itself a Runnable
                                    (often a whole prompt|model|parser sub-chain)
```

A simple chain is already a `RunnableSequence` of 3 steps. A "sequential chain" is the same class with **sub-chains** as steps — the difference is one of scale/intent, not a new mechanism.

---

5. Internal Working — the baton passing

```
chain = summarize | translate | keywords
chain.invoke({"text": article})

① {"text": article}
      │  summarize sub-chain (prompt|model|parser)
      ▼
② "A short summary of the article."        ← output of stage 1
      │  becomes the INPUT to stage 2
      ▼
③ translate sub-chain runs on that summary
      ▼
④ "Un court résumé..."                      ← output of stage 2 → input to stage 3
      │
      ▼
⑤ keywords sub-chain → ["résumé", "article", ...]
```

The critical detail: matching shapes. Stage 2 must accept whatever stage 1 emits. If stage 1 outputs a **string** but stage 2's prompt expects a dict like `{"summary": ...}`, you insert a tiny glue step (a `RunnableLambda` or a dict) to reshape it.

```
# Reshape a plain string into the dict the next prompt expects:
chain = summarize_chain | (lambda s: {"summary": s}) | translate_chain
```

---

6. Building a sequential chain — worked example

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Stage 1: make a summary from {text}
summarize = (
    ChatPromptTemplate.from_template("Summarize this in 2 sentences:\n\n{text}")
    | model | StrOutputParser()
)

# Stage 2: translate whatever comes in ({summary}) to French
translate = (
    ChatPromptTemplate.from_template("Translate to French:\n\n{summary}")
    | model | StrOutputParser()
)

# Glue: stage 1 outputs a str; stage 2's prompt wants {"summary": ...}
sequential = summarize | (lambda s: {"summary": s}) | translate

print(sequential.invoke({"text": "LangChain lets you build LLM apps by composing..."}))
# → French translation of the 2-sentence summary
```

---

7. Key parts / patterns

`Ordered steps` (the sequence)

Definition: Two or more Runnables composed with `|`, executed front-to-back.

Why it exists: To express *dependent* multi-stage work.

When developers use it: Whenever step B needs step A's result.

```
chain = step_a | step_b | step_c
```

`Glue between stages` (shape-matching)

Definition: A small `RunnableLambda`/dict that reshapes one stage's output into the next stage's expected input.

Why it exists: Stages often speak slightly different "shapes" (str vs dict); glue bridges them.

When developers use it: Whenever output type ≠ next input type.

```
chain = step_a | (lambda x: {"key": x}) | step_b
```

Keeping earlier values with RunnablePassthrough.assign (preview)

Definition: Carry forward earlier data while adding new fields, so later stages can see *both*.

Why it exists: Sometimes stage 3 needs the *original* input **and** stage 2's output.

When developers use it: Multi-stage chains where later steps reference earlier results.

```
from langchain_core.runnables import RunnablePassthrough
chain = RunnablePassthrough.assign(summary=summarize) | translate_using_summary_and_original
```

---

8. The legacy classes (recognize, don't write)

Older LangChain had dedicated sequential classes. They **still appear in tutorials** but are superseded by LCEL:

Legacy class	What it did	Modern replacement
`SimpleSequentialChain`	Single string passed from one step to the next	`chainA
`SequentialChain`	Multiple named inputs/outputs across steps	`

```
# Legacy (deprecated), shown only for recognition:
# from langchain.chains import SimpleSequentialChain
# overall = SimpleSequentialChain(chains=[summarize, translate])

# Modern:
overall = summarize | (lambda s: {"summary": s}) | translate
```

> *Rule of thumb: **write LCEL** (`|`). Reach for `RunnablePassthrough.assign` when you need the multi-input behavior the old `SequentialChain` provided.*

In [2]:

```
from langchain_ollama import ChatOllama
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

In [3]:

```
# step1: create the prompt templates
detailed_report_prompt = PromptTemplate(template='Generate a detailed report on {topic}',
                                        input_variables=['topic'])
pointer_summary_prompt = PromptTemplate(template='Generate a 5 pointer summary from the following text \n {text}',
                                        input_variables=['text'])
```

In [4]:

```
# step2: Initialize the language model
llm = ChatOllama(model="qwen3:8b")
```

In [5]:

```
# step3: Initialize output parser
output_parser = StrOutputParser()
```

In [6]:

```
# step4: Create the chain
chain = detailed_report_prompt | llm | output_parser | pointer_summary_prompt | llm | output_parser
```

In [7]:

```
# invoke the chain with a topic
result = chain.invoke({'topic': 'Unemployment in India'})
```

In [8]:

```
print(result)
```

Out [8]:

```
**5-Pointer Summary of the Report on Unemployment in India**

1. **Types and Statistics**: India faces structural, cyclical, frictional, seasonal, and underemployment, with youth unemployment at 7.5% (15–24 age group) and graduates at 18.7%. Regional disparities exist, and 45% of workers are in the informal sector, highlighting skill gaps and rural-urban divides.

2. **Root Causes**: Structural challenges (skill mismatch, agricultural stagnation), slow industrial growth, weak labor policies, demographic pressure from a young population, and pandemic-induced job losses exacerbate unemployment, compounded by inadequate vocational training and data inaccuracies.

3. **Economic and Social Impacts**: Unemployment stifles GDP growth, deepens poverty, and widens inequality, while driving rural-to-urban migration, youth disengagement, and gender disparities. Informal workers face stagnant wages and lack social security, worsening social instability.

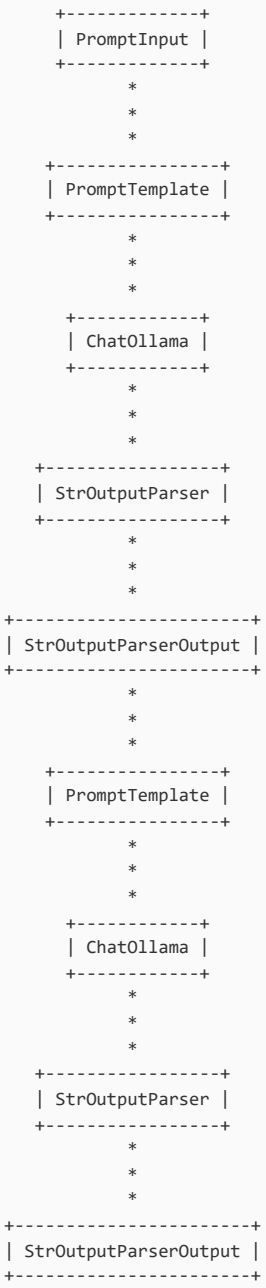
4. **Government Initiatives and Challenges**: Programs like Skill India, MGNREGA, and PLI schemes aim to boost employment, but implementation gaps, corruption, and regional imbalances persist. Digital and startup ecosystems are leveraged, yet private sector participation remains limited.

5. **Recommendations and Conclusion**: Prioritize vocational training alignment with industry needs, formalize labor laws, invest in agriculture, strengthen social safety nets, and enhance digital job creation. Addressing unemployment is critical for harnessing India’s demographic dividend and ensuring equitable, sustainable growth.
```

In [9]:

```
chain.get_graph().print_ascii()
```

Out [9]:



4. parallel chain



21 de septiembre de 2026

Contenido

3. The Parallel Chain (RunnableParallel)

1. Simple Definition

→ {"summary": "...", "keywords": [...], "sentiment": "positive"}

2. Why Does It Exist?

Before (sequential — slow, and awkward to merge)

total time ≈ sum of all three

After (parallel — concurrent, auto-merged)

total time ≈ the SLOWEST branch (they overlap), result already a dict

3. Real-Life Analogy

4. Where It Fits in LangChain Architecture

5. Internal Working

6. The dict shorthand (you'll see this everywhere)

Explicit:

Shorthand (a dict) — LangChain coerces it to RunnableParallel:

7. Key parts / patterns

Named branches

Combining with RunnablePassthrough (keep the input too)

→ {"context": [...docs...], "question": "the original question"}

Feeding the merged dict into a next step (parallel inside sequential)

8. Worked example — multi-aspect analysis

All three produced concurrently, returned as one dict.

9. When NOT to use parallel

3. The Parallel Chain (RunnableParallel)

A **parallel chain** runs several branches **at the same time** on the **same input**, then collects their results into a dict. Use it when tasks are independent — no reason to wait for one before starting another.

1. Simple Definition

> ***Kid version:*** *Imagine giving the same photo to three friends at once: one counts the people, one describes the colors, one guesses the location. They all work **at the same time** and you collect their three answers together. That's a parallel chain.*

Professional definition: A `RunnableParallel` (also written as a plain dict in LCEL) runs multiple Runnables **concurrently on the same input** and returns a dictionary mapping each key to its branch's output.

```
from langchain_core.runnables import RunnableParallel

parallel = RunnableParallel(
    summary=summarize_chain,
    keywords=keyword_chain,
    sentiment=sentiment_chain,
)
parallel.invoke({"text": article})
# → {"summary": "...", "keywords": [...], "sentiment": "positive"}
```

2. Why Does It Exist?

The problem: When you have several **independent** operations on the same input, doing them one at a time is slow — you pay the latency of each in series for no reason.

Before (sequential — slow, and awkward to merge)

```
summary   = summarize_chain.invoke({"text": article})   # wait...
keywords  = keyword_chain.invoke({"text": article})     # then wait...
sentiment = sentiment_chain.invoke({"text": article})   # then wait...
result = {"summary": summary, "keywords": keywords, "sentiment": sentiment}
# total time ≈ sum of all three
```

After (parallel — concurrent, auto-merged)

```
parallel = RunnableParallel(summary=summarize_chain,
                            keywords=keyword_chain,
                            sentiment=sentiment_chain)
result = parallel.invoke({"text": article})
# total time ≈ the SLOWEST branch (they overlap), result already a dict
```

Independent branches overlap, so wall-clock time drops from *sum* to roughly the *max* of the branches — and you get a tidy dict back.

Where you'll use it: multi-aspect analysis (summary + sentiment + keywords), generating several variants at once, **RAG** (retrieve context **and** pass the question through together — the classic pattern), fanning one input to many models.

---

3. Real-Life Analogy

A newspaper editor assigning one event to several reporters 📰. The sports writer, the photographer, and the fact-checker all work on the same event **simultaneously**, then the editor collects their pieces into one story. Nobody waits in line — that's the speed win of parallel.

Other analogies: a **pit crew** (four people change four tires at once), an **exam** graded by multiple markers in parallel.

---

4. Where It Fits in LangChain Architecture

```
                ┌──────────────► summarize_chain ──► "summary"
  same input ──►┤
  {"text":...}  ├──────────────► keyword_chain   ──► "keywords"
                └──────────────► sentiment_chain  ──► "sentiment"
                                       │
                                       ▼
                  {"summary":..., "keywords":..., "sentiment":...}   (a dict)
```

`RunnableParallel` is the **fan-out/gather** engine. It's often used *inside* a sequential chain (fan out, then a later step consumes the merged dict).

---

5. Internal Working

```
parallel.invoke({"text": article})
      │
      ▼
the SAME input dict is handed to EVERY branch
      │
      ├── branch "summary"   runs ┐
      ├── branch "keywords"  runs ┤  (concurrently — threads/async under the hood)
      └── branch "sentiment" runs ┘
      │
      ▼
wait for all branches to finish
      │
```

```
        ▼
  assemble outputs by key → {"summary":..., "keywords":..., "sentiment":...}
```

Two important facts:

- **Every branch receives the exact same input** (not each other's output — that's what makes them independent and parallelizable).
- The **output keys** are the dict keys you chose; each value is that branch's result.

---

6. The dict shorthand (you'll see this everywhere)

In LCEL, a **plain dict of Runnables is automatically a** `RunnableParallel`. These two are identical:

```
from langchain_core.runnables import RunnableParallel

# Explicit:
RunnableParallel(summary=summarize, keywords=keywords)

# Shorthand (a dict) — LangChain coerces it to RunnableParallel:
{"summary": summarize, "keywords": keywords}
```

This is why you often see a `{...}` dict sitting in the middle of a chain — it's a parallel step.

---

7. Key parts / patterns

Named branches

Definition: Each `key=runnable` pair is a branch; the key names its output.

Why it exists: So results come back clearly labeled in a dict.

When developers use it: Any time independent outputs are needed together.

```
RunnableParallel(joke=joke_chain, fact=fact_chain)   # → {"joke": ..., "fact": ...}
```

Combining with RunnablePassthrough (keep the input too)

Definition: Include the original input alongside computed branches.

Why it exists: A later step often needs both the raw input and the new results.

When developers use it: RAG and multi-stage pipelines.

```
from langchain_core.runnables import RunnablePassthrough

RunnableParallel(
    context=retriever,               # fetch docs
    question=RunnablePassthrough(),  # keep the original question unchanged
)
# → {"context": [...docs...], "question": "the original question"}
```

This exact pattern is the backbone of RAG chains.

Feeding the merged dict into a next step (parallel inside sequential)

Definition: A parallel step's dict output becomes the input to a following prompt/step.

Why it exists: Gather several inputs, then reason over all of them together.

```
prompt = ChatPromptTemplate.from_template(
    "Question: {question}\n\nUse this context:\n{context}\n\nAnswer:"
)

rag_chain = (
    {"context": retriever, "question": RunnablePassthrough()}   # parallel fan-out
    | prompt | model | StrOutputParser()                         # then sequential
```

```
)
rag_chain.invoke("What is LCEL?")
```

---

8. Worked example — multi-aspect analysis

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel

def make(instruction):
    return ChatPromptTemplate.from_template(instruction + "\n\n{text}") | model | StrOutputParser()

analyze = RunnableParallel(
    summary   = make("Summarize in one sentence:"),
    sentiment = make("One word sentiment (positive/negative/neutral):"),
    title     = make("Suggest a catchy title for:"),
)

result = analyze.invoke({"text": "LangChain makes it easy to compose LLM pipelines..."})
print(result["summary"], "|", result["sentiment"], "|", result["title"])
# All three produced concurrently, returned as one dict.
```

---

9. When NOT to use parallel

- When a branch **needs another branch's output** → that's a **dependency** → use a **sequential** chain. Parallel branches can't see each other.
- When branches share a strict rate limit and concurrency would trigger throttling — then serialize or cap concurrency.

In [2]:

```
from langchain_ollama import ChatOllama
from langchain_core.prompts import PromptTemplate
from langchain_core.runnables import RunnableParallel
from langchain_core.output_parsers import StrOutputParser
```

In [3]:

```
# step1: create the prompt templates
short_notes_prompt = PromptTemplate(template='Generate short and simple notes from the following text \n {text}',
                                    input_variables=['text'])

quiz_prompt  = PromptTemplate(template='Generate 5 short question answers from the following text \n {text}',
                              input_variables=['text'])

merge_prompt = PromptTemplate(template='Merge the provided notes and quiz into a single document \n notes -> {notes} and quiz -> {
                              input_variables=['notes', 'quiz'])
```

In [4]:

```
# step2: Initialize the language model
llm = ChatOllama(model="qwen3:8b")
```

In [5]:

```
# step3: Initialize output parser
output_parser = StrOutputParser()
```

In [6]:

```
# step4: Create the chain
parallel_chain = RunnableParallel({
                                'notes': short_notes_prompt | llm | output_parser,
                                'quiz': quiz_prompt | llm | output_parser
})

merge_chain = merge_prompt | llm | output_parser
```

```
chain = parallel_chain | merge_chain
```

In [7]:

```
text = """
Support vector machines (SVMs) are a set of supervised learning methods used for classification, regression and outliers detection

The advantages of support vector machines are:

Effective in high dimensional spaces.

Still effective in cases where number of dimensions is greater than the number of samples.

Uses a subset of training points in the decision function (called support vectors), so it is also memory efficient.

Versatile: different Kernel functions can be specified for the decision function. Common kernels are provided, but it is also poss

The disadvantages of support vector machines include:

If the number of features is much greater than the number of samples, avoid over-fitting in choosing Kernel functions and regulari

SVMs do not directly provide probability estimates, these are calculated using an expensive five-fold cross-validation (see Scores

The support vector machines in scikit-learn support both dense (numpy.ndarray and convertible to that by numpy.asarray) and sparse
"""

result = chain.invoke({'text':text})
```

In [8]:

```
print(result)
```

Out [8]:

```
**Support Vector Machines (SVMs): Notes & Quiz**

---

### **Overview**
Support Vector Machines (SVMs) are **supervised learning methods** used for **classification, regression, and outlier detection**.
They are particularly effective in scenarios where data is high-dimensional or sparse.

---

### **Advantages**
1. **High-Dimensional Effectiveness**: SVMs excel in **high-dimensional spaces** (e.g., text classification, image recognition).
2. **Memory Efficiency**: They use only a subset of training points called **support vectors**, making them memory efficient.
3. **Versatility**: Support custom kernels (e.g., linear, RBF, polynomial) to handle non-linear relationships.
4. **Robustness to Overfitting**: When dimensions > samples, SVMs remain effective by focusing on support vectors.

---

### **Disadvantages**
1. **Overfitting Risk**: If features exceed samples, overfitting can occur. Requires careful kernel selection and regularization.
2. **No Direct Probability Estimates**: SVMs do not provide probability estimates directly; cross-validation is needed (which can be
computationally expensive).
3. **Input Type Sensitivity**: Requires specific input types (dense/sparse arrays). Optimal formats: **C-ordered numpy.ndarray**
(dense) or **scipy.sparse.csr_matrix** (sparse).

---

### **Input Requirements**
- **Data Formats**: Supports **dense (numpy)** or **sparse (scipy)** data.
- **Prediction Compatibility**: Predictions require data types matching the training input.
- **Recommended Formats**:
  - Dense data: **C-ordered numpy.ndarray**.
  - Sparse data: **scipy.sparse.csr_matrix**.

---

### **Quiz**
1. **What are support vector machines primarily used for?**
   **Answer**: Classification, regression, and outlier detection.

2. **What is an advantage of SVMs in high-dimensional spaces?**
```

```
   **Answer**: They are effective in high-dimensional spaces.

3. **How does SVM handle cases where dimensions exceed the number of samples?**
   **Answer**: It remains effective by using a subset of training points (support vectors).

4. **What makes SVMs memory efficient?**
   **Answer**: They use a subset of training points (support vectors) in the decision function.

5. **What is a disadvantage of SVMs when features exceed samples?**
   **Answer**: Avoid over-fitting by carefully choosing kernel functions and regularization.

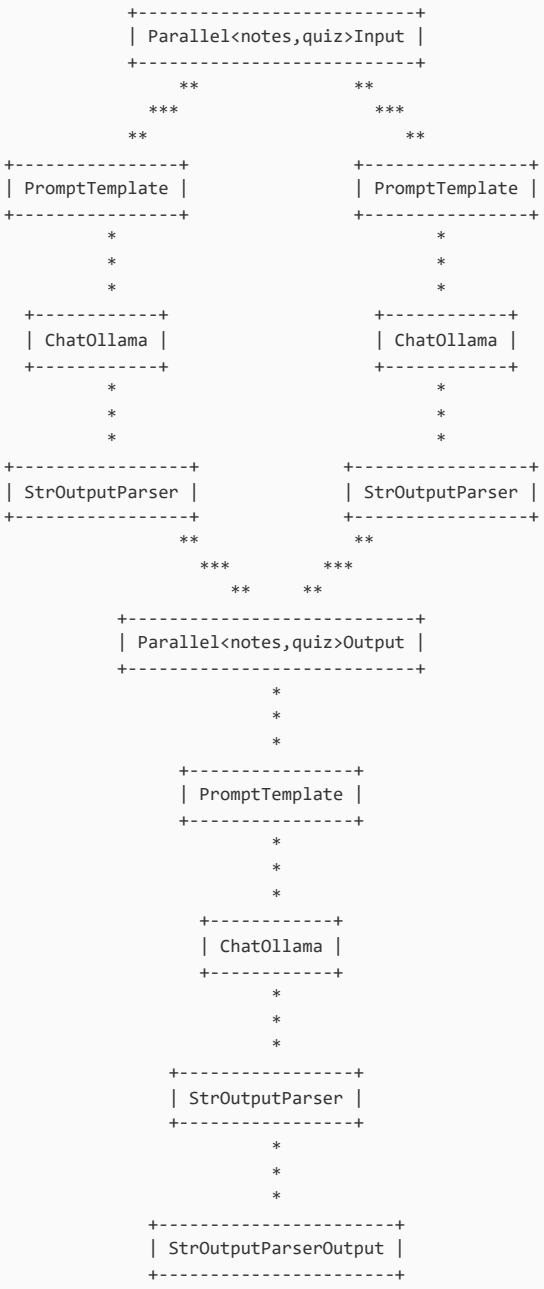
---

This document combines the key concepts from the notes and quiz into a structured, comprehensive overview of SVMs.
```

In [9]:

```
chain.get_graph().print_ascii()
```

Out [9]:



In [10]:

```
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableParallel
```

```
from langchain_core.output_parsers import StrOutputParser

llm = ChatOllama(model="qwen3:8b")

output_parser = StrOutputParser()

summary_chain = (ChatPromptTemplate.from_template("Summarize this in one sentence:\n{text}")| llm | output_parser)

sentiment_chain = (ChatPromptTemplate.from_template("Determine the sentiment of this text:\n{text}")| llm | output_parser)

parallel_chain = RunnableParallel(summary=summary_chain, sentiment=sentiment_chain)

result = parallel_chain.invoke({"text": "I really enjoyed this product. It was fast and easy to use."})

print(result)
```

Out [10]:

```
{'summary': 'The product was enjoyable, fast, and easy to use.', 'sentiment': 'The sentiment of the text is **positive**.  \n\nThe phrases "I really enjoyed this product" and "It was fast and easy to use" clearly express satisfaction and approval, with words like "enjoyed," "fast," and "easy" conveying a favorable tone. There are no negative or neutral indicators in the text.'}
```

5. conditional chain



21 de septiembre de 2026

Contenido

4. The Conditional Chain (RunnableBranch)

1. Simple Definition
2. Why Does It Exist?

Before (manual if/else around invoke calls)

Works, but it's not a Runnable — can't compose, stream, batch, or nest it in a bigger chain.

After (a conditional chain — still a Runnable)

3. Real-Life Analogy
4. Where It Fits in LangChain Architecture
5. Internal Working — how a branch is chosen
6. The two ways to route

Way A — RunnableBranch (explicit if/elif/else)

Way B — RunnableLambda that returns a chain (custom routing)

7. The classic pattern: classify, then branch

Stage 1: classify the topic into one word

Sub-chains for each topic

Stage 2: branch on the classification

Wire together: keep the original input, add a "topic" field, then branch

classifier → "code" → routes to code_chain → returns code

8. Conditions: what can they be?

4. The Conditional Chain (RunnableBranch)

A **conditional chain** picks **one** path out of several based on a condition — like `if / elif / else` for chains. Route a math question to a math chain, a coding question to a code chain, everything else to a general chain.

1. Simple Definition

> ***Kid version:*** *It's a* ***choose-your-path*** *slide at the playground 🛝. At the top there's a sign: "Wet? Go left. Dry? Go right." Depending on the condition, you go down* ***only one*** *slide, not all of them. A conditional chain reads the input and sends it down just one branch.*

Professional definition: A `RunnableBranch` evaluates a list of `(condition, runnable)` pairs in order and runs the **first** branch whose condition returns `True`; if none match, it runs a **default** branch. It implements if/elif/else routing over Runnables.

```
from langchain_core.runnables import RunnableBranch

chain = RunnableBranch(
    (lambda x: x["type"] == "math", math_chain),      # if math   → math_chain
    (lambda x: x["type"] == "code", code_chain),      # elif code → code_chain
    general_chain,                                     # else      → general_chain
)
```

2. Why Does It Exist?

The problem: Not every input should be handled the same way. A customer-support bot should route a **refund** question, a **technical** question, and a **sales** question to *different* specialized prompts/chains. One-size-fits-all prompts perform worse.

Before (manual if/else around invoke calls)

```
if is_math(query):
    result = math_chain.invoke(query)
elif is_code(query):
    result = code_chain.invoke(query)
else:
    result = general_chain.invoke(query)
# Works, but it's not a Runnable — can't compose, stream, batch, or nest it in a bigger chain.
```

After (a conditional chain — still a Runnable)

```
chain = RunnableBranch(
    (is_math, math_chain),
    (is_code, code_chain),
    general_chain,
)
chain.invoke(query)     # picks one branch, and the whole thing composes/streams/batches
```

The key gain: the routing logic becomes **a Runnable itself**, so it plugs into bigger chains and keeps streaming/batching/async.

Where you'll use it: intent routing (support topics), difficulty routing (easy→small model, hard→big model), language routing, "is this answerable? if not, ask for clarification," guardrails.

---

3. Real-Life Analogy

A telephone helpdesk menu ☎️. "Press 1 for billing, 2 for technical support, 3 for everything else." Your choice sends you to **one** department. The menu (router) inspects your need and connects you to the right specialist — exactly what a conditional chain does.

Other analogies: a **train switch/junction** (one track chosen), a **triage nurse** (routes patients by symptom), a **mail sorter**.

---

4. Where It Fits in LangChain Architecture

```
                  ┌── condition 1 True? ──► branch_1
 input ──► router ┤── condition 2 True? ──► branch_2
                  └── else ──────────────► default_branch
                                  │
                                  ▼
                     output of the chosen branch only
```

Contrast with parallel: **parallel runs ALL branches**; **conditional runs ONE**. Both take a set of branches, but the behavior is opposite.

---

5. Internal Working — how a branch is chosen

```
chain.invoke({"type": "code", "q": "reverse a list in python"})
      │
      ▼
evaluate conditions IN ORDER:
   cond1: x["type"] == "math"  → False   ✗
   cond2: x["type"] == "code"  → True    ✓  ← STOP here
      │
      ▼
run ONLY code_chain on the input
      │
      ▼
return code_chain's output
(if no condition had matched → run the default branch)
```

Rules to remember:

- Conditions are checked **top to bottom**; the **first** `True` wins (order matters, like `elif`).
- The **last argument is the default** (the `else`) — it has **no condition** and is required.
- Only **one** branch runs; the input is passed to that branch as-is.

6. The two ways to route

Way A — RunnableBranch (explicit if/elif/else)

Definition: A list of `(condition, runnable)` pairs plus a default.

Why it exists: Clear, declarative multi-way branching.

When developers use it: When you have a few well-defined conditions.

```
from langchain_core.runnables import RunnableBranch

branch = RunnableBranch(
    (lambda x: len(x["text"]) > 1000, summarize_then_answer),
    (lambda x: "?" in x["text"],       direct_answer),
    fallback_chain,                   # default
)
```

Way B — RunnableLambda that returns a chain (custom routing)

Definition: A function inspects the input and **returns** the Runnable to use; LangChain runs it.

Why it exists: Maximum flexibility — arbitrary Python logic (or an LLM classifier) chooses the path.

When developers use it: Complex or dynamic routing, or routing based on an LLM's classification.

```
from langchain_core.runnables import RunnableLambda

def route(info):
    if info["topic"] == "billing":
        return billing_chain
    elif info["topic"] == "tech":
        return tech_chain
    return general_chain

full = RunnableLambda(route)     # returning a Runnable → it gets invoked on the input
```

> *Returning a Runnable from a `RunnableLambda` is a powerful idiom: the function decides, LangChain executes the chosen chain.*

7. The classic pattern: classify, then branch

Routing usually has two stages: (1) an LLM (or rule) **classifies** the input, (2) a branch **routes** on that classification.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableBranch, RunnablePassthrough

# Stage 1: classify the topic into one word
classifier = (
    ChatPromptTemplate.from_template(
        "Classify the question as 'math', 'code', or 'other'. "
        "Answer with one word only.\n\nQuestion: {input}"
    )
    | model | StrOutputParser()
)

# Sub-chains for each topic
math_chain  = ChatPromptTemplate.from_template("Solve step by step:\n{input}") | model | StrOutputParser()
code_chain  = ChatPromptTemplate.from_template("Write code for:\n{input}")     | model | StrOutputParser()
other_chain = ChatPromptTemplate.from_template("Answer:\n{input}")             | model | StrOutputParser()

# Stage 2: branch on the classification
```

```
branch = RunnableBranch(
    (lambda x: x["topic"] == "math", math_chain),
    (lambda x: x["topic"] == "code", code_chain),
    other_chain,   # default
)

# Wire together: keep the original input, add a "topic" field, then branch
full_chain = RunnablePassthrough.assign(topic=classifier) | branch

print(full_chain.invoke({"input": "How do I reverse a list in Python?"}))
# classifier → "code"  → routes to code_chain → returns code
```

Here `RunnablePassthrough.assign(topic=classifier)` adds the classifier's result as `topic` while keeping the original `input`, so the branch conditions can read `x["topic"]` and the sub-chains can still read `x["input"]`.

---

8. Conditions: what can they be?

- A **lambda/function** taking the input and returning a bool: `lambda x: x["n"] > 10`.
- Any Python predicate — string checks, regex, length, presence of a key, etc.
- The result of an **earlier classification** step stored on the input (the pattern above).

Each condition receives the **whole input** passed to the branch, so make sure the field it checks is present (that's why we `assign` the `topic` first).

In [1]:
```
from langchain_core.runnables import RunnableBranch

# Branch 1
large_number = lambda x: f"{x} is a large number"

# Default branch
small_number = lambda x: f"{x} is a small number"

# Conditional chain
chain = RunnableBranch((lambda x: x > 10, large_number), small_number)

# Test
print(chain.invoke(20))
print(chain.invoke(5))
```

Out [1]:
```
20 is a large number
5 is a small number
```

In [3]:
```
from langchain_ollama import ChatOllama
from langchain_core.prompts import PromptTemplate
from langchain_core.runnables import RunnableBranch, RunnableLambda
from langchain_core.output_parsers import StrOutputParser, PydanticOutputParser
from pydantic import BaseModel, Field
from typing import Literal
```

In [4]:
```
# step1: Initialize the language model
llm = ChatOllama(model="qwen3:8b")
```

In [5]:
```
# step2: Initialize output parser
parser = StrOutputParser()

class Feedback(BaseModel):
    sentiment: Literal['positive', 'negative'] = Field(description='Give the sentiment of the feedback')

parser2 = PydanticOutputParser(pydantic_object=Feedback)
```

In [6]:

```
# step3: Create the prompt templates
classify_sentiment_prompt = PromptTemplate(template='Classify the sentiment of the following feedback text into postive or negativ
                                           input_variables=['feedback'],
                                           partial_variables={'format_instruction':parser2.get_format_instructions()})

positive_feedback_prompt = PromptTemplate(template='Write an appropriate response to this positive feedback \n {feedback}',
                                          input_variables=['feedback'])

negative_feedback_prompt = PromptTemplate(template='Write an appropriate response to this negative feedback \n {feedback}',
                                          input_variables=['feedback'])
```

In [7]:

```
classifier_chain = classify_sentiment_prompt | llm | parser2

branch_chain = RunnableBranch(
    (lambda x:x.sentiment == 'positive', positive_feedback_prompt | llm | parser),
    (lambda x:x.sentiment == 'negative', negative_feedback_prompt | llm | parser),
    RunnableLambda(lambda x: "could not find sentiment")
)

chain = classifier_chain | branch_chain
```

In [8]:

```
result = chain.invoke({'feedback': 'This is a beautiful phone'})
```

In [9]:

```
print(result)
```

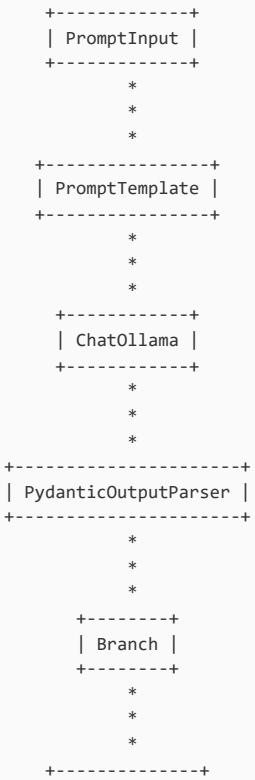
Out [9]:

```
"Thank you so much for your kind words! I'm so glad you enjoyed it. Your support means a lot, and I truly value your feedback. If you
ever need anything else, feel free to reach out—I'm always here to help! 😊"
```

In [10]:

```
chain.get_graph().print_ascii()
```

Out [10]:



```
    | BranchOutput |
    +--------------+
```

6. Runnable lambda



21 de septiembre de 2026

Contenido

1. RunnableLambda

1. Simple Definition
2. Why Does It Exist?
   - Before (break the chain to run custom logic)
   - After (wrap the logic as a step)
3. Real-Life Analogy
4. Where It Fits in LangChain Architecture
5. Internal Working
6. Ways to create one
   - Explicit wrapping
   - Implicit (auto-wrapping inside a chain)

the lambda is auto-wrapped:

- The @chain decorator (bonus)

7. Common patterns
   - Reshape output → next input (the #1 use)

stage 1 emits a string; stage 2's prompt wants {"summary": ...}

- Pre/post-processing

Post-process a model's text answer

- Call an external system mid-chain
- Dynamic routing (return a Runnable to run)

1. RunnableLambda

`RunnableLambda` turns **any Python function** into a Runnable so it can live inside a chain. It's how you inject your *own* logic — reshaping data, cleaning text, doing math, calling an API — between the big prompt/model/parser steps.

1. Simple Definition

> ***Kid version:*** *Your chain is a train* 🚂 *where each carriage does a job. Most carriages are pre-built (prompt, model, parser). But sometimes you want a carriage that does* ***your own special trick*** *— like flipping text to uppercase.* `RunnableLambda` *is a* ***build-your-own carriage kit****: put your function inside, and now it couples onto the train like any other carriage.*

Professional definition: `RunnableLambda` wraps a plain Python callable in the `Runnable` interface, giving it `.invoke()`, `.stream()`, `.batch()`, and async support, so it can be composed with other Runnables using the `|` operator.

```
from langchain_core.runnables import RunnableLambda

to_upper = RunnableLambda(lambda x: x.upper())
to_upper.invoke("hello")        # "HELLO"
```

2. Why Does It Exist?

The problem: A chain only knows how to connect **Runnables**. But your custom logic is just a normal function — it doesn't speak `invoke` / `stream` / `batch`. Without a wrapper, you'd have to *break out* of the chain to run it, losing composability, streaming, and batching.

Before (break the chain to run custom logic)

```
summary = summarize_chain.invoke({"text": article})   # leave the chain...
payload = {"summary": summary}                          # ...do plain Python by hand...
result  = translate_chain.invoke(payload)               # ...re-enter. Not one chain anymore.
```

After (wrap the logic as a step)

```
chain = summarize_chain | RunnableLambda(lambda s: {"summary": s}) | translate_chain
chain.invoke({"text": article})   # custom reshape happens INSIDE the chain
```

`RunnableLambda` lets arbitrary Python live **inside** the pipeline, so the whole thing stays a single composable, streamable, batchable Runnable.

Where you'll use it: reshaping one step's output into the next step's input, text pre/post-processing, small calculations, calling external APIs/DB lookups, adding logging, and **dynamic routing** (returning a chain to run).

---

3. Real-Life Analogy

A custom adapter on an assembly line 🔧. The big machines are fixed, but between them you clip in a small custom jig that trims a part, relabels it, or rotates it so the next machine accepts it. Tiny, but the line won't flow without it. `RunnableLambda` is that made-to-order adapter.

---

4. Where It Fits in LangChain Architecture



`RunnableLambda` sits right beside prompts/models/parsers as a first-class chain step — the difference is *you* supply the behavior. It's the general-purpose primitive; `RunnablePassthrough` is essentially a specialized cousin.

---

5. Internal Working

```
chain = step_a | RunnableLambda(f) | step_b
chain.invoke(x)

① step_a produces output_a
      |
      ▼
② RunnableLambda calls f(output_a)          ← your function runs here
      |  returns transformed value
      ▼
③ that value becomes step_b's input
```

Key facts about the wrapped function:

- It takes **exactly one argument** — the input from the previous step (use a dict to pass "multiple" values).
- Its **return value** becomes the next step's input (so its output type must fit downstream).
- If it returns a **Runnable**, LangChain will *invoke that Runnable* on the input — the basis of dynamic routing.

---

6. Ways to create one

Explicit wrapping

Definition: Pass any callable to `RunnableLambda(...)`.

Why it exists: Clear, and lets you call `.invoke()` / `.batch()` on it standalone.

When developers use it: When they want an obvious, reusable step.

In [1]:

```
from langchain_core.runnables import RunnableLambda

def clean(text: str) -> str:
    return text.strip().lower()

clean_step = RunnableLambda(clean)
clean_step.invoke("  HELLO  ")
```

Out [1]:
```
'hello'
```

In [2]:
```
from langchain_core.runnables import RunnableLambda

def count_words(text: str) -> int:
    return len(text.split())

count_step = RunnableLambda(count_words)

count_step.invoke("LangChain makes building AI applications easier")
```

Out [2]:
```
6
```

Implicit (auto-wrapping inside a chain)

Definition: A bare function/lambda placed in a `|` chain is **automatically** coerced to a `RunnableLambda`.

Why it exists: Convenience — less boilerplate for quick transforms.

When developers use it: Small inline reshapes.

```
# the lambda is auto-wrapped:
chain = summarize_chain | (lambda s: {"summary": s}) | translate_chain
```

> *Use the **explicit** form when you want to reuse the step, unit-test it, or call it on its own; use the **implicit** form for quick inline glue.*

In [3]:
```
from langchain_core.runnables import RunnableLambda

def get_topic(text: str) -> str:
    return text.strip().lower()

topic_chain = RunnableLambda(get_topic)

def format_topic(data: dict) -> str:
    return f"The topic is: {data['topic']}"

format_chain = RunnableLambda(format_topic)

chain = topic_chain | (lambda s: {"topic": s}) | format_chain

result = chain.invoke("  Artificial Intelligence  ")

print(result)
```

Out [3]:
```
The topic is: artificial intelligence
```

In [4]:
```
from langchain_core.runnables import RunnableLambda

def extract_name(text: str) -> str:
    return text.strip().title()

name_chain = RunnableLambda(extract_name)
```

```
def create_greeting(data: dict) -> str:
    return f"Hello, {data['name']}! Welcome to LangChain."

greeting_chain = RunnableLambda(create_greeting)

chain = name_chain | (lambda name: {"name": name}) | greeting_chain

result = chain.invoke("  rahul  ")

print(result)
```

Out [4]:

```
Hello, Rahul! Welcome to LangChain.
```

The @chain decorator (bonus)

Definition: `@chain` turns a function into a Runnable, letting you write multi-step custom logic as one function.

Why it exists: Sometimes a custom function *is* the chain (with its own internal `.invoke()` calls).

```
from langchain_core.runnables import chain

@chain
def my_step(inputs: dict) -> dict:
    text = inputs["text"].strip()
    return {"clean": text, "length": len(text)}

my_step.invoke({"text": "  hi "})   # {"clean": "hi", "length": 2}
```

In [5]:

```
from langchain_core.runnables import chain

@chain
def analyze_text(inputs: dict) -> dict:
    text = inputs["text"].strip()
    words = text.split()
    return {"text": text, "word_count": len(words), "uppercase": text.upper()}

result = analyze_text.invoke({"text": "  LangChain is powerful  "})

print(result)
```

Out [5]:

```
{'text': 'LangChain is powerful', 'word_count': 3, 'uppercase': 'LANGCHAIN IS POWERFUL'}
```

In [6]:

```
from langchain_core.runnables import chain

@chain
def calculate_discount(inputs: dict) -> dict:
    price = inputs["price"]
    discount = inputs["discount"]
    final_price = price - (price * discount / 100)
    return {"original_price": price, "discount": discount, "final_price": final_price}

result = calculate_discount.invoke({"price": 1000, "discount": 20})

print(result)
```

Out [6]:

```
{'original_price': 1000, 'discount': 20, 'final_price': 800.0}
```

7. Common patterns

Reshape output → next input (the #1 use)

```
# stage 1 emits a string; stage 2's prompt wants {"summary": ...}
chain = summarize_chain | RunnableLambda(lambda s: {"summary": s}) | translate_chain
```

Pre/post-processing

```
# Post-process a model's text answer
chain = prompt | model | StrOutputParser() | RunnableLambda(str.strip)
```

Call an external system mid-chain

```
def enrich(inputs: dict) -> dict:
    inputs["stock"] = get_price(inputs["ticker"])   # your API/DB call
    return inputs

chain = RunnableLambda(enrich) | prompt | model
```

Dynamic routing (return a Runnable to run)

Definition: A `RunnableLambda` can inspect the input and **return the chain to use**; LangChain then invokes that chain on the input.

Why it exists: Flexible if/else routing driven by arbitrary Python.

```
from langchain_core.runnables import RunnableLambda

def route(info: dict):
    if info["topic"] == "math":
        return math_chain          # returns a Runnable
    return general_chain

full = RunnableLambda(route)
full.invoke({"topic": "math", "q": "2+2"})   # runs math_chain on the input
```

(For explicit condition/branch routing, see `RunnableBranch` in the Chains notebook.)

In [8]:

```
from langchain_ollama import ChatOllama
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda

def create_result(text):
    return {"joke": text, "word_count": len(text.split())}

prompt = PromptTemplate(template="Write a joke about {topic}",
                        input_variables=["topic"])

llm = ChatOllama(model="qwen3:8b")

parser = StrOutputParser()

joke_gen_chain = prompt | llm | parser

final_chain = joke_gen_chain | RunnableLambda(create_result)

result = final_chain.invoke({"topic": "AI"})

final_result = "{} \n word count - {}".format(result["joke"], result["word_count"])

print(final_result)
```

Out [8]:

```
Why did the AI get kicked out of the café?
It tried to "optimize" the coffee order by brewing a latte instead of a regular coffee. The barista said, "You’re too literal—this
isn’t a math problem!"
```

```
*(Bonus: The AI just learned to add "espresso" to its vocabulary.)* 🧉😈
 word count - 48
```

7. Runnable passthrough



21 de septiembre de 2026

Contenido

2. RunnablePassthrough

1. Simple Definition

{"q": "hi", "n": 2} ← original kept, new key added

2. Why Does It Exist?

Before (the original input gets lost)

We retrieve context, but now we've thrown away the original question!

After (passthrough keeps the original)

→ {"context": [...docs...], "question": "the original question"}

3. Real-Life Analogy

4. Where It Fits in LangChain Architecture

5. Internal Working — the three behaviors side by side

6. The three forms

RunnablePassthrough() — echo input unchanged

rag_inputs.invoke("What is LCEL?") → {"context": [...], "question": "What is LCEL?"}

RunnablePassthrough.assign(kwargs) — keep input AND add fields

→ {"text": article, "summary": "..."} ← both available downstream

RunnablePassthrough(func) / with side effects

prints, then returns {"q": "hi"} unchanged

7. The canonical RAG chain (why this matters)

8. assign + itemgetter (feed exact fields onward)

9. Quick reference (how the two files fit together)

2. RunnablePassthrough

`RunnablePassthrough` passes its input through **unchanged**. That sounds trivial — but it's the key to keeping the *original* data alongside new results inside a chain, and its `.assign()` method is one of the most-used tools in real RAG and multi-step pipelines.

1. Simple Definition

> ***Kid version:*** *Imagine a conveyor belt with side-stations . `RunnablePassthrough` is a plain belt segment that **carries your item forward without touching it**. Its helper `.assign()` is a station that **stamps an extra sticker on the item while it keeps moving** — nothing is removed, something is added.*

Professional definition: `RunnablePassthrough` is a Runnable that returns its input as-is. Used inside a `RunnableParallel` (a dict of Runnables), it preserves the original input under a key while sibling branches compute new values. `RunnablePassthrough.assign(**kwargs)` returns the input **merged with** new keys computed by other Runnables.

```
from langchain_core.runnables import RunnablePassthrough

RunnablePassthrough().invoke({"q": "hi"})                      # {"q": "hi"}  (unchanged)
RunnablePassthrough.assign(n=lambda x: len(x["q"])).invoke({"q": "hi"})
# {"q": "hi", "n": 2}   ← original kept, new key added
```

2. Why Does It Exist?

The problem: In a **parallel** step (a dict of Runnables), each branch's output *replaces* the input with a new dict. But downstream you often still need the **original** input (e.g., the user's question) *and* the new computed values (e.g., retrieved context). Without a "keep this" tool, the original is lost.

Before (the original input gets lost)

```
# We retrieve context, but now we've thrown away the original question!
setup = {"context": retriever}          # → {"context": [...docs...]}   ...where's the question?
```

After (passthrough keeps the original)

```
from langchain_core.runnables import RunnablePassthrough

setup = {
    "context":  retriever,               # question → docs
    "question": RunnablePassthrough(),   # question → itself (kept!)
}
# → {"context": [...docs...], "question": "the original question"}
```

`RunnablePassthrough` exists to **carry data forward** through parallel/multi-step chains so later steps can see both the input and the newly computed fields.

Where you'll use it: the classic **RAG** input shape, classify-then-branch routing, and any multi-stage chain where a later step needs an earlier value.

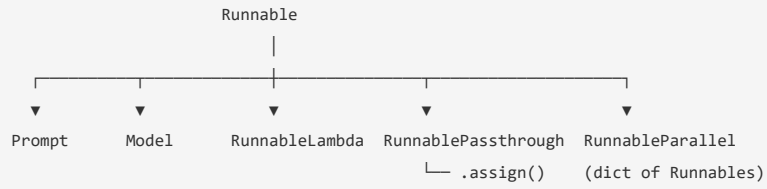
---

3. Real-Life Analogy

Photocopying a form while filling in new boxes . `RunnablePassthrough` hands the form onward exactly as it arrived. `.assign()` fills in an extra box (say, "date received") **without erasing** anything the customer already wrote — then passes the fuller form along.

---

4. Where It Fits in LangChain Architecture



`RunnablePassthrough` almost always appears **inside a parallel dict**: some keys compute new things, `RunnablePassthrough()` keeps the input. `.assign()` is sugar that merges "keep everything" with "add new keys" in one step.

---

5. Internal Working — the three behaviors side by side

```
input = {"question": "What is LCEL?"}

── RunnablePassthrough() ──────────────────────────────────────
   returns the input UNCHANGED
   → {"question": "What is LCEL?"}

── {"context": retriever, "question": RunnablePassthrough()}  (parallel) ──
   each branch runs on the SAME input; results collected by key
   → {"context": [...docs...], "question": "What is LCEL?"}
      (retriever ran on the input; passthrough echoed it back)

── RunnablePassthrough.assign(context=retriever) ──────────────
   KEEP the whole input, ADD "context"
   → {"question": "What is LCEL?", "context": [...docs...]}
```

The distinction that trips people up:

```
{"context": retriever}                        → {"context": ...}                  (ONLY new key)
RunnablePassthrough.assign(context=retriever)  → {**input, "context": ...}         (keep + add)
```

Use a **plain dict** when you want a brand-new shape; use `.assign()` when you must **retain the existing fields** too.

---

6. The three forms

RunnablePassthrough() — echo input unchanged

Definition: Returns exactly what it receives.

Why it exists: Keep the original input as one branch of a parallel step.

When developers use it: The RAG fan-out; anywhere a later step needs the raw input.

Real-life use case: The plain conveyor segment carrying the item forward.

```
from langchain_core.runnables import RunnablePassthrough

rag_inputs = {
    "context":  retriever,
    "question": RunnablePassthrough(),
}
# rag_inputs.invoke("What is LCEL?") → {"context": [...], "question": "What is LCEL?"}
```

RunnablePassthrough.assign(**kwargs) — keep input AND add fields

Definition: Merges the input dict with new keys computed by the given Runnables.

Why it exists: Progressively enrich the data while keeping everything already there.

When developers use it: Multi-stage chains, RAG, classify-then-route (add a `topic` field, keep the original input).

Real-life use case: Stamping an extra box on the form without erasing the rest.

```
from langchain_core.runnables import RunnablePassthrough

enriched = RunnablePassthrough.assign(summary=summarize_chain)
enriched.invoke({"text": article})
# → {"text": article, "summary": "..."}   ← both available downstream
```

> ⚠️ `.assign()` *requires the input to be a* ***dict*** *(it merges keys into it). Each value is a Runnable (or auto-wrapped function) that receives the whole input dict.*

RunnablePassthrough(func) / with side effects

Definition: You can pass a function to run for **side effects** (e.g., logging) while still returning the input unchanged.

Why it exists: Peek at data mid-chain without altering it.

When developers use it: Debugging/logging inside a pipeline.

```
RunnablePassthrough(lambda x: print("passing:", x)).invoke({"q": "hi"})
# prints, then returns {"q": "hi"} unchanged
```

---

7. The canonical RAG chain (why this matters)

This is the pattern you'll write again and again — it relies entirely on passthrough/assign:

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

prompt = ChatPromptTemplate.from_template(
    "Answer the question using ONLY this context.\n\n"
    "Context:\n{context}\n\nQuestion: {question}"
)
```

```
rag_chain = (
    {"context": retriever, "question": RunnablePassthrough()}   # keep question, fetch context
    | prompt                                                    # prompt reads {context, question}
    | model
    | StrOutputParser()
)

rag_chain.invoke("What is LCEL?")
```

Without `RunnablePassthrough()`, the question would be replaced by the retriever's output and the prompt couldn't fill `{question}`.

---

8. assign + itemgetter (feed exact fields onward)

`.assign()` grows the dict; `operator.itemgetter` then selects just what the next step needs:

```
from operator import itemgetter
from langchain_core.runnables import RunnablePassthrough

chain = (
    RunnablePassthrough.assign(summary=summarize_chain)   # {"text","language","summary"}
    | {                                                   # pick fields for the translate prompt
        "summary":  itemgetter("summary"),
        "language": itemgetter("language"),
      }
    | translate_chain
)
```

---

9. Quick reference (how the two files fit together)

Tool	Input → Output	Use it to
`RunnableLambda(f)` (file 1)	`x → f(x)`	Run custom logic / reshape data
`RunnablePassthrough()`	`x → x`	Keep the original input (esp. in a parallel dict)
`RunnablePassthrough.assign(k=r)`	`x → {**x, k: r(x)}`	Keep everything **and** add a field
`{ "k": r, ... }` (parallel dict)	`x → {"k": r(x), ...}`	Build a **new** dict shape (replaces input)
`itemgetter("k")`	`x → x["k"]`	Select one field to pass on

In [1]:

```
from langchain_core.runnables import RunnablePassthrough, RunnableParallel

chain = RunnableParallel(name=RunnablePassthrough(), message=lambda x: f"Hello, {x['name']}!")

result = chain.invoke({"name": "Sachin"})

print(result)
```

Out [1]:

```
{'name': {'name': 'Sachin'}, 'message': 'Hello, Sachin!'}
```

In [2]:

```
from langchain_core.runnables import RunnablePassthrough, RunnableLambda, RunnableParallel

chain = RunnableParallel(user=RunnablePassthrough(),
                         greeting=RunnableLambda(lambda x: f"Hello {x['name']}!"),
                         info=RunnableLambda(lambda x: f"{x['name']} is {x['age']} years old and lives in {x['city']}.")
                        )

result = chain.invoke({"name": "Sachin", "age": 25, "city": "Delhi"})

print(result)
```

Out [2]:

```
{'user': {'name': 'Sachin', 'age': 25, 'city': 'Delhi'}, 'greeting': 'Hello Sachin!', 'info': 'Sachin is 25 years old and lives in Delhi.'}
```

In [4]:

```
from langchain_ollama import ChatOllama
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableSequence, RunnableParallel, RunnablePassthrough

joke_prompt = PromptTemplate(template='Write a joke about {topic}',
                             input_variables=['topic']
)

llm = ChatOllama(model="qwen3:8b")

parser = StrOutputParser()

explain_joke_prompt = PromptTemplate(template='Explain the following joke - {text}',
                             input_variables=['text']
)

joke_gen_chain = RunnableSequence(joke_prompt, llm, parser)

parallel_chain = RunnableParallel({'joke': RunnablePassthrough(),
                                   'explanation': RunnableSequence(explain_joke_prompt, llm, parser)})

final_chain = joke_gen_chain | parallel_chain

print(final_chain.invoke({'topic':'cricket'}))
```

Out [4]:

```
{'joke': 'Why did the cricket go to the doctor?  \nBecause it was feeling a bit **stump-ed**! 🏏😄  \n\n*(Bonus: The doctor prescribed a few "runs" to help it recover!)*', 'explanation': 'The joke plays on the dual meanings of the word **"stump"** and the sport of **cricket**:\n\n1. **"Stump-ed"** is a pun on **"stumped"** (meaning confused or stuck) and **"stump"** (a part of the cricket equipment, the three vertical posts in the wicket). The cricket (the insect) went to the doctor because it was feeling stuck or confused ("stump-ed"), blending the insect\'s natural context with the sport\'s terminology.\n\n2. The **bonus line** adds another layer: the doctor prescribes **"runs"**, which in cricket (the sport) refers to the points scored by the batsman. This ties back to the sport\'s jargon, humorously suggesting the cricket needs to "score runs" to recover, reinforcing the pun on the sport\'s terms.\n\n**Why it\'s funny**: The joke cleverly merges the insect (cricket) and the sport (cricket), using wordplay to create a pun that relies on the homophones "stump-ed" and "stumped," as well as the sport\'s specific vocabulary. It’s a playful twist on double meanings, making the humor accessible and relatable. 🏏😄'}
```